NEBIUS X NVIDIA GLOBAL AI HACKATHON

ShipCheck Build Plan

Full Pipeline, Resources & Day-by-Day Schedule
Coding and Agentic Engineering Track

Version
1.0 · October 5, 2026

Status
Locked for build

Team
3 builders · Oct 6 – Oct 30, 2026

Submission deadline
October 30, 2026, 10:00 AM PT
(10:30 PM IST)

Build window
19 working days

Judging
December 1–15, 2026

What this is: The step-by-step execution plan for ShipCheck — the adversarial verification gate for AI-generated code with a rectification CD loop. Companion to the ShipCheck PRD (architecture, requirements, demo script).

Contents

1. How to use this document & roles
2. Part A — Resources (A1 hackathon-provided, A2 beyond the hackathon)
3. Part B — Reference builds: what to copy, what to avoid
4. Part C — Demo fixtures
5. Part D — Day-by-day schedule (Weeks 1–4, Days 1–19)
6. Part E — Risk register
7. Appendix — Key links

How to use this document

Each day has one goal, per-person tasks, and a done-criteria — the definition of "done" that evening. If a day slips, the fallback is listed; weekends are buffer, not planned work. Companion doc: **ShipCheck PRD** (architecture, requirements, demo script).

> **Build window:** October 6 → October 30, 2026 (19 working days). **Submission deadline:** October 30, 2026, 10:00 AM PT (10:30 PM IST). **Judging:** December 1–15, 2026.

Roles

Person	Ownership
P1 — Sandboxes, probes & infra	Contree adapter, probe image, PoC engine, Serverless deploy.
P2 — Models, pipeline & rectification	Nemotron client, attack planner, fix loop, risk score, Tavily grounding.
P3 — Surface & ship	Dashboard, GitHub PR bot, README, video, submission.

Operating principle from the PRD: reserve Nemotron 3 Ultra for planning + hard fixes; let Super / Nano handle fast, everyday calls. Budget and tier mapping are in Part A.

PART A · RESOURCES

A1. Provided by the hackathon (claim on Day 1)

#	Resource	What you get	How to claim	Lead time
1	Token Factory credits (promo)	**$25** TF credits	Form at nebius.com/promo-code, activation code `NEBIUS-DEVPOST-GLOBAL26`	Minutes
2	Nebius AI Builder Program (free)	**$25** TF credits + **$25** Tavily credits (both expire 90 days after issuance) + $100 LangSmith + $50 Toloka + $50 Tandem + $1 Academy cert + office hours with Nebius engineers	dev.nebius.com/builders — email double opt-in (no disposable emails)	Hours (opt-in)
3	Tavily free tier (backup)	1,000 credits/month	tavily.com signup	Minutes
4	Nebius Discord	Direct access to Nebius engineers + hackathon channel	Link on Devpost Resources tab / Builder Program welcome email (verify — old invites may be stale)	Minutes
5	Docs & cookbooks	token-factory-cookbook (agentic coding recipes), docs.tokenfactory.nebius.com, docs.contree.dev, docs.nebius.com/serverless	Bookmarked Day 1	—
6	Builders & Brews events	$100 TF + $100 AI Cloud + Tavily credits for attendees, workshops, mentors	luma.com/builderandbrews — RSVP if a nearby city works	Days
7	Rate-limit headers	`x-ratelimit-remaining-*` on TF responses for adaptive backoff	In API responses	—

Budget math: $50 TF credits total covers the build if Ultra is reserved for planning + hard fixes (per the PRD's tier mapping). Sandbox ops are beta — budget conservatively.

Source: hackathon Resources page supplied by the team (nebiusglobalaihackathon.devpost.com/resources). Verify every claim link on Day 1 — see Appendix.

PART A · RESOURCES

A2. Required beyond hackathon provisions

Must-have ($0)

- GitHub organization + repo (private → public on Day 17); classic PAT (`repo` scope) as primary, fine-grained PAT (Contents + Pull requests: write) as the least-privilege story; a **second GitHub account** to play the "human reviewer" who merges the draft PR in the demo.
- Tavily backup key (free tier) in case Builder Program credits are slow.
- YouTube account (unlisted upload for the demo video).
- Python 3.11+, Docker, Node 20, git on every dev machine.
- Binaries: `semgrep`, `gitleaks`, `osv-scanner`, `pip-audit` (pre-baked into the sandbox image anyway).
- OBS (screen recording) + DaVinci Resolve (free video editing).
- Fixture forks: fork `kaizenmantra/vulnshop` into the team org on Day 2 (primary demo target: ~60-line Flask, SQLi + command injection + reflected XSS); add a 10-line path-traversal route to the fork.

Lead-time items (do Day 1 morning)

Builder Program double opt-in; promo-form credit claim; Discord join; Builders & Brews RSVP if relevant.

Nice-to-have

$100 LangSmith credit (trace one scan end-to-end — good judging artifact); $1 Academy cert; CapCut premium (faster editing than Resolve).

> **Expected total cost: $0.** Optional: $5–15/month only if you want a custom domain/registry (not needed).

PART B

Reference builds: what to copy, what to avoid

Project	Copy	Avoid
sandforge (Contree + Nemotron PR agent, same hackathon)	Provider interfaces (`NebiusLlm` / `MockLlm` , `ContreeSandbox` / `LocalSandbox`) so all three can develop offline before cloud keys; checkpoint-branching mental model (`disposable:false` → `result_image_uuid` = branch point); Nano-cheap / Super-quality model split	They hand-rolled the HTTP client (no TS SDK) — you're on Python, use `contree-sdk` pinned behind an adapter
archon (Contree + Nemotron, same hackathon)	Model router by stage (Ultra diagnosis → Super orchestration → Nano compaction); **Tavily grounding before reasoning**; fork-one-sandbox-per-attempt; **replay-mode demo** — recorded golden runs stream through the dashboard so judges never depend on a live call	Their stack is heavier (Next.js 16 + Monaco + xterm) than 3 people need — keep the dashboard lean, put the wow in the evidence timeline
Google CodeMender (GA Oct 2026)	The validated loop: scan → PoC exploit in sandbox → diff patch → LLM-as-a-judge → **human approval, never auto-commit**; agent role split (Classifier → Investigator → Validator → Fixer) as Nemotron prompt boundaries	Don't replicate the enterprise governance story — your differentiator is the cheap/fast open-stack version
petriqore/pentest-agent	Finding dedupe keyed on (endpoint + root cause); run-artifact persistence (`reports/run-<ts>/`); `--max-iterations` budgets (mirrors your 50-op cap); "No PASS, No PR" gate	Fully agentic free-roam — keep your probes deterministic with boolean markers
Project Naptime (Google Project Zero)	"Perfect verification" (crash-or-no-crash = your `uid=` / `root:` markers); multiple independent trajectories (your asyncio fan-out); a Controller that aborts stalled probes (your per-probe timeouts)	Targets C/memory-corruption with debuggers — you need HTTP-level probes
Semgrep/gitleaks/osv-scanner CI canon	`semgrep scan --config p/auto --config p/owasp-top-ten --sarif --metrics=off || true` ; gitleaks with `fetch-depth: 0` ; `pip-audit -r requirements.txt --format json` ; SARIF everywhere; warm the ruleset cache at image-build time	Don't gate twice on secrets; scope history scans to the PR diff

PART C

Demo fixtures

Target	Why	Verdict
vulnshop (`kaizenmantra/vulnshop` , forked)	~60 lines Flask; SQLi `POST /login` , command injection `GET /export` , reflected XSS `GET /search` ; ships with Semgrep action + a reference fix branch	**Primary.** Matches 3 of 4 classes 1:1
Team-added traversal route	10-line Flask route in your fork	Covers class 4 deterministically
VulnSocial / vulnmart	Backup Flask fixtures, richer surface	Backups only
DVWA (`vulnerables/web-dvwa`)	Real app; **security-level toggle (low→impossible) is a demo superpower**	Stretch “real app” segment only
Juice Shop / WebGoat	Too heavy / mismatched	Skip

Determinism rules: pin every image tag; fixtures live in your fork; record the actual boolean markers as the visual proof.

PART D

Day-by-day schedule — operating rhythm

- **15-min standup daily** (async ok): shipped yesterday / today / blockers. Blockers older than 4 hours get pair-swapped.
- **Merge to main daily.** No feature branches living past 48 hours.
- **Friday demo ritual:** live run for the team, recorded blockers, retro notes.
- **Weekends are buffer**, not planned work. Feature freeze Oct 26 — only P0 fixes after.

Three HIGH RISK days: Day 2 (sandbox image + adapters), Day 5 (PoC engine), Day 11 (Serverless deploy). Each has a named fallback so no single failure blocks the team.

PART D · WEEK 1

Week 1 — Foundations (Oct 6–9)

Day 1 — Tue Oct 6 · Goal: keys, credits, smoke tests

- **P1:** claim $25 TF credits (promo form + `NEBIUS-DEVPOST-GLOBAL26`); join Builder Program (double opt-in) for +$25 TF / +$25 Tavily; `pip install contree-sdk` (pinned); smoke test `images().run(shell="echo ok")`; run the PRD §13 verification checklist (pricing page, function-calling test, egress test) and log results.
- **P2:** Tavily key (Builder Program + free-tier backup); test `TavilyClient.search`; test Nemotron Nano with the JSON-in-prompt pattern; confirm the `reasoning_content` fallback works.
- **P3:** GitHub org + repo (private); classic PAT (`repo` scope) + fine-grained PAT draft; second "reviewer" account; YouTube channel; OBS + DaVinci installed.

Done: `.env` with all keys (never committed); smoke-test log in repo; §13 checklist with pass/fail per item.

Risk / Fallback: Credit/opt-in delays → start on free tiers; nothing blocks on credits Day 1.

Day 2 — Wed Oct 7 · Goal: sandbox image + adapters HIGH RISK

- **P1:** build `shipcheck-probe` image (Python 3.11, Node 20, semgrep with warmed ruleset cache, gitleaks, pip-audit, osv-scanner, flask, requests) → checkpoint UUID in config; `SandboxClient` adapter (spawn/run/poll/artifacts, `Semaphore(40)`, client-side timeouts); `MockLlm` + `LocalDockerSandbox` fallback (sandforge pattern).
- **P2:** `NemotronClient` (JSON extraction, `reasoning_content` fallback, `force_nonempty_content`, retries, rate-limit backoff); role presets per tier.
- **P3:** fork vulnshop into team org; add 10-line traversal route; verify all 4 vuln classes fire locally; pin versions.

Done: Image UUID recorded; adapters unit-tested against mocks; fixture fork runs locally with 4 confirmed vulns.

Risk / Fallback: If the Contree image build fails → local Docker PoC runner + mocks; nobody is blocked.

PART D · WEEK 1

Week 1 (continued)

Day 3 — Thu Oct 8 · Goal: triage chain

- **P1:** triage runner in `SandboxClient` : gitleaks → pip-audit/npm-audit + osv-scanner → Semgrep `p/owasp-top-ten` ; SARIF → finding JSON.
- **P2:** finding schema `{{id, class, file, line, severity_hint, evidence}}` ; dedupe key (rule + file + line-range).
- **P3:** verdict report renderer v0 (JSON → Markdown); FastAPI dashboard skeleton.

Done: vulnshop fixture → triage JSON containing the expected findings.

Depends on: Day 2 image + adapters.

Day 4 — Fri Oct 9 · Goal: attack planning + W1 checkpoint

- **P2:** Ultra attack-planner prompt (Classifier/Investigator roles); JSON-in-prompt; Tavily exploit-writeup grounding for the detected stack.
- **P1:** wire planner output → probe queue.
- **P3: Friday demo ritual #1** — live triage + attack plan on vulnshop; record blockers; W1 retro.

Done: Attack-plan JSON for vulnshop with 4+ planned attacks; retro notes filed.

Depends on: Day 3 triage.

Friday demo ritual #1: live triage + attack plan on vulnshop, watched together. Record blockers; file W1 retro notes the same evening.

PART D · WEEK 2

Week 2 — The Loop (Oct 12–16)

Day 5 — Mon Oct 12 · Goal: PoC engine HIGH RISK

- **P1:** probe harness — one fork per finding, app + attacker in one run chain, fixed ports, readiness polling, per-probe timeouts; SQLi + command-injection probes.
- **P2:** traversal + XSS probes (LLM-generated payload scripts); boolean marker verdict logic.
- **P3:** evidence capture (JSON blob + raw logs as downloadable artifacts); dashboard probe-status view.

Done: 4/4 markers reproduce on fixtures, stable across 3 consecutive runs.

Risk / Fallback: Forked vulnshop + recorded evidence; drop to 3 classes if traversal stays flaky.

Day 6 — Tue Oct 13 · Goal: Tavily grounding + risk score

- **P2:** Tavily CVE/advisory grounding per confirmed finding; risk engine (severity × PoC-confirmed escalator × reachability × exposure, 0–100, one-sentence factor explanations).
- **P1:** reachability analysis (sink reachable from an entrypoint — Semgrep taint or LLM call graph).
- **P3:** verdict report UI with ranked findings.

Done: vulnshop report shows CVE links + scores with explanations.

Depends on: Day 5 probes.

PART D · WEEK 2

Week 2 (continued)

Day 7 — Wed Oct 14 · Goal: rectification loop

- **P2:** fix generator (diff-hunk JSON from exploit evidence, applied programmatically); LLM-as-a-judge re-verification (PoC re-run in child branch + repo's own tests).
- **P1:** child-branch mechanics (checkpoint after patch; re-run probe in the branch).
- **P3:** "No PASS, No PR" gate state in the UI.

Done: SQLi fixture fixed end-to-end: PoC red → patch → PoC green → tests pass.

Day 8 — Thu Oct 15 · Goal: GitHub PR bot

- **P3:** push `shipcheck/fix-<class>-<slug>` branch; `POST /pulls` with `draft: true`; dedupe by branch name + finding fingerprint (update body if a PR exists); evidence PR body template (summary, exploitability, PoC + marker, Tavily links, score + why, diff, verification, reviewer checklist, "ShipCheck never auto-merges").
- **P2:** fix prompts for the remaining 3 classes.
- **P1:** budget guard (`--max-iterations` style; 50-op cap accounting).

Done: Draft PR open on the fixture fork with a full evidence body; a second run updates it instead of duplicating.

PART D · WEEK 2 → WEEK 3

End of Week 2 & start of Week 3

Day 9 — Fri Oct 16 · Goal: first end-to-end + dashboard v1

- **All:** integration day. P3 ships dashboard v1 (scan input → live probe status → verdict → evidence → fix diff → PR link).
- **Friday demo ritual #2** — full pipeline on vulnshop, watched together; W2 retro; **cut list finalized** (whatever isn't working is dropped, in writing).

Done: One-click scan → verdict → fix → PR from the dashboard.

Cut list rule: at the Week 2 retro, anything not working is dropped in writing. Protect the core loop: attack → verdict → fix → re-verify → draft PR.

Week 3 — Harden, Deploy, Record (Oct 19–23)

The focus shifts from building the loop to making it reliable, public, and filmable: real-repo shakedown, Serverless deploy with replay mode, golden run, script, and video production.

PART D · WEEK 3

Week 3 — Harden, Deploy, Record

Day 10 — Mon Oct 19 · Goal: real-repo shakedown

- **P1:** run the pipeline on 1–2 small real Python repos with known vulns; fix flakiness found.
- **P2:** false-positive policy (PoC-unconfirmed → score ~0, no PR, shown as “not exploitable”).
- **P3:** README draft (setup, run, model IDs, where TF/Nemotron/Tavily accelerated the work).

Done: Real-repo run documented; FP policy implemented.

Day 11 — Tue Oct 20 · Goal: Serverless deploy HIGH RISK

- **P1:** containerize FastAPI + dashboard; push to Nebius registry; `nebius ai endpoint create --public --auth token --container-port 8000` ; strip the `/32` from the public IP; poll to RUNNING; warm the URL; **replay mode** (archon pattern): golden-run replay + recorded evidence so the judges’ experience never depends on a live call.
- **P2/P3:** support + README polish.

Done: Public URL serving the dashboard; replay mode working.

Risk / Fallback: Debug the CPU endpoint; replay-mode dashboard carries the demo regardless.

PART D · WEEK 3

Week 3 (continued)

Day 12 — Wed Oct 21 · Goal: golden run

- **All:** full pipeline run on the vulnshop fixture; capture and freeze golden artifacts; rehearse the 4 demo beats (problem → attack lands → fix → PR).

Done: Golden artifacts committed; beats timed to fit 3 minutes.

Day 13 — Thu Oct 22 · Goal: video script + README

- **P3:** 3-minute script (PRD §10) + shot list; README final (badges, architecture diagram, model table, Tavily section).
- **P1/P2:** fix anything the golden run exposed. No new features.

Done: Script approved by all three; README complete.

PART D · WEEK 3 → WEEK 4

Week 3 close & Week 4 opening

Day 14 — Fri Oct 23 · Goal: video production

- **P3:** record screen + voiceover. **P1/P2:** quiet hardening only — **no feature work after lunch**.
- **Friday demo ritual #3:** watch the rough cut together; note fixes.

Done: Rough cut exists.

Week 4 — Buffer & Submit (Oct 26–30)

Feature freeze is in force. Week 4 is video final, judging-criteria pass, submission prep, a full 24-hour-early rehearsal, and submit — with a hard internal deadline well before the official one.

Feature freeze Oct 26: only P0 fixes after this date. Protect P3's video days — they are the least movable work in the plan.

PART D · WEEK 4

Week 4 — Buffer & Submit (Oct 26–30)

Day 15 — Mon Oct 26 · Goal: video final + feature freeze

- **P3:** edit, captions, upload unlisted to YouTube; verify < 3 min and that the audio covers Token Factory + Nemotron usage.
- **P1/P2:** P0 bugfixes only.

Done: Final video URL; repo tagged `v1.0`.

Day 16 — Tue Oct 27 · Goal: judging-criteria pass

- **All:** explicit pass over the four criteria — Implementation (TF + Nemotron load-bearing?), Design (coherent product?), Impact (problem slide lands?), Idea (non-obvious?). Fix gaps found.
- Draft the required **written feedback** on Nebius Token Factory / AI Cloud / NVIDIA tooling.

Done: Criteria checklist signed off; feedback text written.

PART D · WEEK 4

Week 4 (final days)

Day 17 — Wed Oct 28 · Goal: submission prep

- **P3:** flip repo public; add Apache-2.0 LICENSE at top level; complete the Devpost draft (description, track, repo link, demo URL, video link, feedback text).
- **P1/P2:** final live-URL check; keep the demo warm.

Done: Devpost draft 100% complete — not submitted yet.

Day 18 — Thu Oct 29 · Goal: 24-hour-early rehearsal

- **All:** fresh-machine README test (clone → setup → run); video watch-through; demo URL from incognito; fix anything found. **Do not submit** — this day is the buffer.

Done: Submission-ready; known-issues list reviewed.

Day 19 — Fri Oct 30 · Goal: SUBMIT

- Morning (IST): final checks; **submit by 9:00 AM PT (9:30 PM IST) at the latest** — deadline is 10:00 AM PT / 10:30 PM IST.
- Confirm submission receipt; announce in Discord.

Done: Submitted. Keep the demo URL live through judging (Dec 1–15).

PART E

Risk register (day-level)

Day	Risk	Fallback
Day 2	Contree image build / SDK drift blocks everyone	Local Docker PoC runner + `MockLlm`; adapters keep the interfaces stable
Day 5	Fixture nondeterminism (ports, timing)	Forked vulnshop + recorded evidence; cut to 3 vuln classes
Day 11	Serverless deploy fails / cold starts	Replay-mode dashboard; CPU endpoint debug with `nebius ai endpoint logs`
Day 1	Credit / opt-in delays	Free tiers (Tavily 1k/mo, TF trial); nothing blocks Day 1
Any	Nemotron quirk (empty content, no tools)	JSON-in-prompt + `reasoning_content` fallback (already the default design)
Any	A person is unavailable	Daily merges + small interfaces mean any two can cover; P3's video days are the least movable — protect them

APPENDIX

Key links (verify Day 1)

- Devpost hackathon page + Resources tab (credits form, Discord invite)
- dev.nebius.com/builders (Builder Program)
- github.com/nebius/token-factory-cookbook · docs.tokenfactory.nebius.com · docs.contree.dev · docs.nebius.com/serverless
- github.com/nebius/contree-sdk · github.com/nebius/contree-mcp
- Fixture: github.com/kaizenmantra/vulnshop (fork it) · backups: VulnSocial, vulnmart · stretch: `vulnerables/web-dvwa`
- Reference builds: github.com/antrikshh90/sandforge · github.com/codewitheugene/archon

Uncertainties carried from research: exact Contree Sandbox pricing (beta) — budget conservatively; current Discord invite link — pull from Devpost Resources tab; Nemotron per-token pricing — verify on the Token Factory console before heavy Ultra use.

Hackathon resources page as supplied: nebiusglobalaihackathon.devpost.com/resources — re-verify all claim forms, codes, and invite links on Day 1 morning before any build work depends on them.